computers

MDPI

Article

Heterogeneous Graph Transformer with Multi-View Representation Learning for Flaky Test Detection

Peng Dai [1], Xiaoqin Ma [1,*], Yanyang Zhao [2] and Yunzhan Gong [3]

[1] School of Big Data and Artificial Intelligence, Chizhou University, Chizhou 247100, China
[2] BNRist, Tsinghua University, Beijing 100084, China
[3] State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China
* Correspondence: mxq@czu.edu.cn

Abstract

Continuous Integration pipelines rely on large-scale automated testing to support rapid releases. However, flaky tests exhibit non-deterministic outcomes under an identical code and configuration, substantially increasing rerun costs and hindering fault localization. Existing approaches struggle to uniformly model heterogeneous runtime evidence and its multi-relational structure in CI environments, which limits cross-project generalization and interpretability. To address this gap, this paper presents HgtFlaky, a runtime-evidence-centered multi-view heterogeneous graph learning framework. A Unified Event Model is introduced to normalize heterogeneous CI artifacts into semantically consistent event quadruples, and a heterogeneous execution graph is then constructed to capture testing entities and multiple relation types. Based on the HEG, three complementary views are derived to characterize run-level, test-level, and thread-level flaky behaviors. A heterogeneous graph Transformer is further adopted to jointly encode the multi-view graph instances and learn transferable test-level representations for flaky/non-flaky prediction. Experiments on two benchmark datasets, FlakeFlagger and IDoFT, show that HgtFlaky achieves strong and stable performance. Under 10-fold cross-validation, it obtains an F1-score of 83% on FlakeFlagger and 98% on IDoFT. Under per-project validation on FlakeFlagger, HgtFlaky achieves 78% Precision, 89% Recall, and 81% F1-score, outperforming Flakify by 8 percentage points and FlakeFlagger by 74 percentage points in F1-score.

Keywords: Continuous Integration; runtime evidence; heterogeneous execution graph; Unified Event Model; test flakiness

Check for updates

Academic Editors: Waddah Saeed and AbdulRahman Alsewari

Received: 9 May 2026
Revised: 29 May 2026
Accepted: 3 June 2026
Published: 7 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Software systems have become critical infrastructure across domains such as finance, manufacturing, transportation, and e-government, making software quality and stability a central concern in modern software engineering [1]. A recent report from the Consortium for Information & Software Quality (CISQ) estimates that software defects cause more than USD 2.41 trillion in direct economic losses each year, a substantial portion of which arises from faults that are not detected and removed during testing [2]. These figures highlight that improving software quality and engineering efficiency is not only essential for individual systems, but also fundamental to sustaining the growth of the digital economy and large-scale digital transformation.

With the rapid evolution of software development practices, Continuous Integration (CI) and Continuous Delivery (CD) have become mainstream engineering practices in

modern software organizations [3]. CI pipelines rely heavily on large-scale automated testing to safeguard quality and stability under rapid iteration. However, flaky tests remain a long-standing challenge for both industry and academia [4,5]. A flaky test is typically defined as one that exhibits non-deterministic outcomes under nominally unchanged code and configuration, i.e., the same test may pass in some executions but fail in others. Prior empirical and industrial studies have shown that flaky tests are pervasive in large-scale software systems [6]. For example, a longitudinal study of 55 Java projects reported that about 75% of flaky tests were already flaky at the time of their introduction [4]. Industrial observations further indicate that distinguishing true regressions from flaky failures can consume 2–16% of overall testing budgets due to repeated reruns, while also incurring additional waiting time and compute costs [7,8].

Existing flaky-test detection techniques mainly rely on repeated executions, static or dynamic analysis, feature-based learning, or code/text-based prediction [9–18]. Although these methods are useful in specific settings, they often suffer from three limitations in CI environments. First, multi-source runtime artifacts such as logs, stack traces, call traces, and resource events are usually modeled separately or transformed into coarse-grained features, making it difficult to preserve their temporal and semantic relationships [12,15,19]. Second, many learning-based methods treat each test as an isolated sample and fail to explicitly represent heterogeneous runtime entities and their multi-relational dependencies [13,20–23]. Third, prediction results are often returned as single confidence scores, providing limited evidence for understanding why a test is classified as flaky [20–22]. These limitations motivate a unified and relation-aware representation of CI runtime evidence for robust and interpretable flaky-test detection.

To address these limitations, this paper presents HgtFlaky, an evidence-driven framework that models flaky behavior as a multi-view heterogeneous execution graph (HEG). A Unified Event Model (UEM) is first constructed to normalize and align multi-source CI artifacts into a semantically consistent event space. Based on UEM, a HEG is built with typed nodes and typed relations, and three complementary views are derived to capture flakiness patterns at the run, test, and thread levels. A shared heterogeneous graph Transformer then jointly encodes these views and produces a representation for flaky prediction, supporting both cross-project robustness and evidence-aware interpretation. For clarity, the main abbreviations and symbols used in this paper are summarized in Abbreviations.

The main contributions are summarized as follows.

- A UEM and HEG construction pipeline is developed to normalize and align multi-source CI artifacts into a unified, relation-aware representation.
- HgtFlaky is presented as a runtime-evidence-driven flaky-test detector that represents CI executions as multi-view HEGs and leverages a heterogeneous graph Transformer for prediction.
- Extensive experiments are conducted on two public benchmarks, FlakeFlagger and IDoFT, under both cross-validation and cross-project settings, demonstrating that HgtFlaky consistently outperforms representative baselines and that multi-view modeling yields stable gains.

This paper addresses the following research questions:

- **RQ1:** How effective is HgtFlaky for flaky-test detection?
- **RQ2:** How does HgtFlaky compare with state-of-the-art flaky-test detectors?
- **RQ3:** Does multi-view HEG modeling improve flaky-test detection performance?

The remainder of this paper is organized as follows. Section 2 reviews the related work. Section 3 presents the proposed flaky-test detection method in detail. Section 4 describes

the experimental setup and discusses the experimental results. Section 5 discusses threats to the validity of this study. Finally, Section 6 concludes the paper.

2. Related Work

With the widespread adoption of CI, flaky tests have evolved into a costly quality concern [5]. Flaky tests exhibit non-deterministic outcomes under a nominally unchanged code and configuration, directly undermining the trustworthiness of regression testing, increasing pipeline latency, and incurring additional compute and human triage costs. A recent survey of software developers reports that about 59% of respondents have to deal with flaky tests on a monthly, weekly, or even daily basis [24], forcing teams to repeatedly discriminate between real regressions and noisy test failures. Beyond productivity and developer experience, flakiness also reduces the reliability of downstream techniques that assume stable test signals, such as mutation testing and automated program repair, and can distort data-driven decision-making in CI.

Prior work has investigated flaky tests from multiple perspectives. One major line of research systematically characterizes their causes, costs, and CI-level impact [25]. It identifies common root causes such as order dependencies, timing and asynchrony, concurrency scheduling, and external environment factors. Large-scale interviews and questionnaire studies further suggest that reproduction and localization remain core challenges in flaky-test management, and that CI resource constraints and scheduling policies can exacerbate the manifestation of flakiness [26–32].

A second major line of work focuses on flaky-test detection and prediction. Early approaches primarily relied on repeated re-execution [33] and rule-based static analysis, and later expanded to dynamic instrumentation, machine learning-based predictors [6], and log-driven statistical models, as well as specialized ranking techniques for specific patterns [34–36]. In parallel, researchers have examined the extent to which flaky tests can be detected without executing tests, i.e., using purely static information. Multi-project empirical studies indicate that classifiers trained solely on static metrics, such as complexity measures, test smells, and dependency structure, can yield practically useful results, yet their cross-project generalization and explanatory granularity remain limited [23]. Along this line, recent work represents tests with data-dependency graphs and applies graph neural networks, improving over baselines such as FlakeFlagger on more than 17,500 test cases and identifying high-risk flaky tests without running them [22]. Prior feature-level analyses indicate that dynamic runtime signals are informative for flaky-test detection, but their effectiveness is often affected by cross-project distribution shifts and preprocessing choices [19,37].

A third major line of work targets flaky-test reproduction and repair [38–42]. Research along this direction has moved beyond merely triggering flaky behavior toward achieving stable replay and producing more actionable and explainable repairs. To this end, existing techniques leverage explicit modeling of event and thread schedules, search for minimal failure-inducing sequences, and mine differential invariants. For instance, FlakeSync [39] automatically identifies critical sections and synchronization barriers and injects lightweight synchronization primitives with minimal changes to business logic. It achieves high repair success rates while incurring lower runtime overhead than manual or heuristic remedies.

Finally, flakiness research is expanding beyond conventional code-centric analysis to encompass multi-language ecosystems as well as emerging application domains. The First Flaky-Test Workshop (FTW 2024) [26] highlighted shared priorities in both industry and academia, including mitigating flaky failures in CI, enabling earlier detection and automated repair, and addressing randomness and non-determinism in AI-enabled systems. Prior work suggests that, even when executions are expensive, combining limited exe-

cutions with machine learning-based classifiers can still provide effective flakiness assessment [43,44]. Meanwhile, automated test generation tools may themselves introduce additional flakiness due to internal randomness and non-idempotent behaviors, posing new challenges for traditional flaky-test detection and mitigation pipelines [45].

In summary, existing studies have advanced flaky-test research from root-cause characterization, detection and prediction to reproduction and repair. However, most existing detectors still rely on repeated executions, handcrafted static or dynamic features, code-text representations, or coarse-grained runtime statistics. These methods are often effective in specific settings, but they provide limited support for jointly modeling heterogeneous runtime entities and their multi-relational dependencies in CI environments. Compared with feature-based methods and language-model-based methods, HgtFlaky differs in both evidence representation and representation learning. It normalizes multi-source runtime artifacts through UEM, constructs HEGs with typed nodes and relations, and further derives run-level, test-level, and thread-level views to capture complementary flaky-test evidence. Therefore, the novelty of HgtFlaky lies not simply in applying a heterogeneous graph Transformer, but in formulating flaky-test detection as multi-view heterogeneous runtime-evidence modeling for evidence-aware prediction.

3. HgtFlaky

3.1. Problem Formulation and Overall Framework

This work aims to automatically determine whether a test case is flaky or non-flaky by leveraging multi-source runtime evidence collected in CI environments. Formally, let $\mathcal{T}$ denote the set of test cases in a project. For each test $t \in \mathcal{T}$, the CI pipeline may execute it multiple times, yielding a set of test runs $\{r_{t,1}, r_{t,2}, \dots\}$. Each run is accompanied by heterogeneous runtime artifacts. Given labeled tests in the training set, our goal is to learn a predictor f that maps the collected runtime evidence to a flakiness label and generalizes across projects and software versions. To this end, this paper presents HgtFlaky, a heterogeneous graph Transformer with multi-view representation learning, as overviewed in Figure 1.

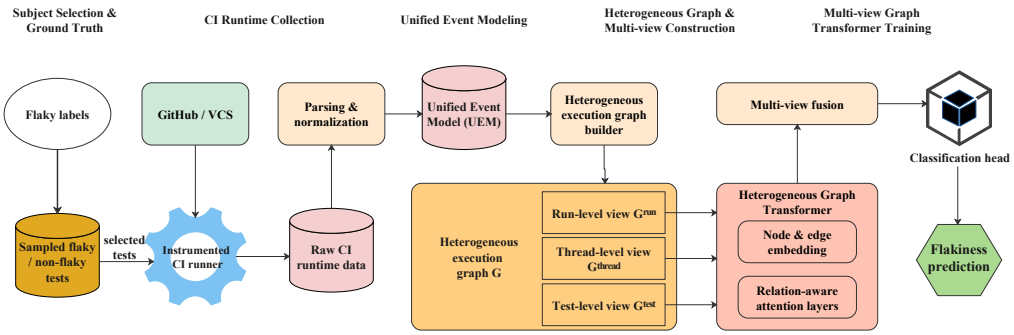


Figure 1. Architecture of the HgtFlaky. Multiple views derived from the same execution are encoded by shared Transformer layers, aggregated into a test-level embedding, and optimized with supervised classification and contrastive objectives.

To clarify the design rationale of HgtFlaky, we reformulate the framework as a staged pipeline rather than a collection of independent modules. First, the UEM normalizes heterogeneous CI artifacts, including logs, call traces, exception stacks, and resource events, into temporally aligned event tuples. Second, the HEG maps these events to typed runtime entities and typed relations, explicitly representing dependencies among test executions, threads, resources, exceptions, and code symbols. Third, multi-view slicing derives run-level, test-level, and thread-level subgraphs from the same HEG, allowing the model to

capture complementary flakiness evidence at different granularities. Finally, a shared heterogeneous graph Transformer encodes these typed subgraphs with relation-aware attention and fuses their representations for flaky/non-flaky prediction. The model is optimized with supervised classification and multi-view contrastive regularization.

3.2. UEM

In real CI environments, a single test execution is accompanied by runtime signals collected from multiple channels, including log messages produced by the test framework and the system under test, method-call traces, exception stack traces, and resource monitoring signals. These sources are highly heterogeneous in format, granularity, and temporal resolution. As a result, performing feature engineering directly on raw records makes it difficult to align temporal and causal relationships across sources and capture interaction patterns among runtime entities in a structured manner. To address this challenge, Hgt-Flaky introduces a UEM that maps logs, call traces, exceptions, and monitoring records into a unified event representation, serving as a semantically consistent and learnable intermediate layer for subsequent HEG construction. As illustrated in Figure 2, the UEM transforms multi-source runtime artifacts into time-aligned events and then maps them to CI semantic entities and relations, yielding a CI semantic graph that explicitly encodes runtime entities and their interactions.

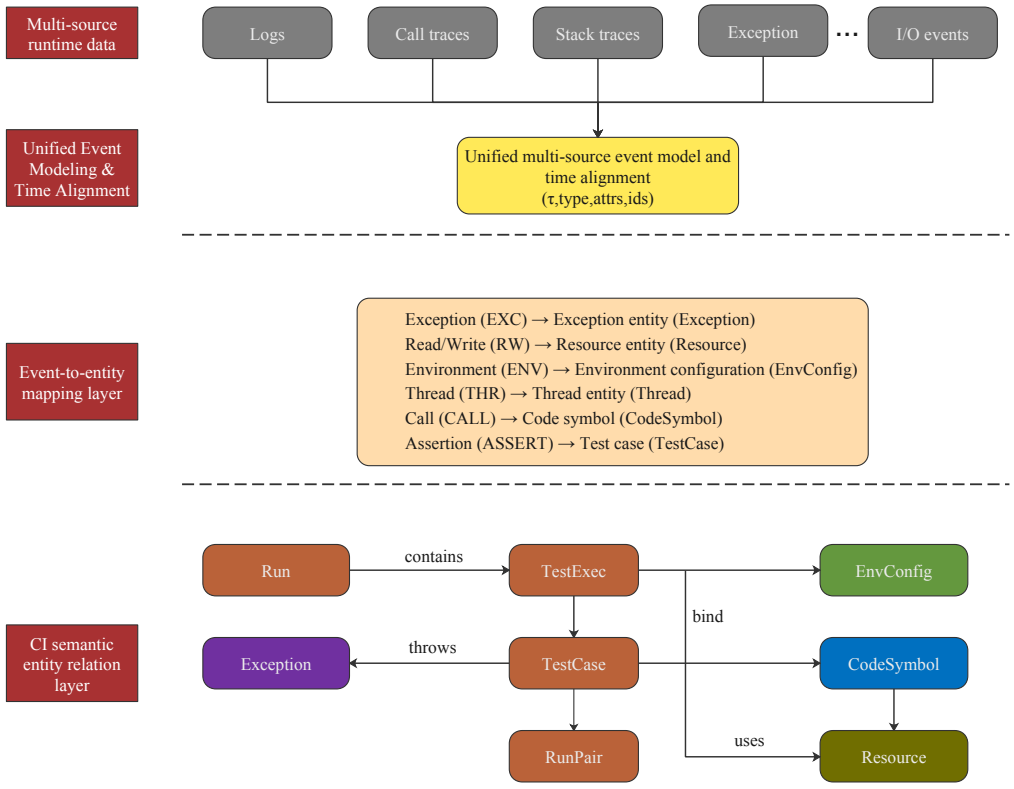


Figure 2. Unified Event Model for CI runtime data. The model normalizes multi-source runtime signals into unified events and maps them to CI semantic entities and relations.

In the UEM, each record observed during a single test run is mapped to a unified event unit, represented as a quadruple $e = \langle t, type, attrs, ids \rangle$. Here, t is a logical timestamp that totally orders events within a run and supports deriving a conservative happens-before relation, capturing temporal precedence and potential causal ordering. The field *type* specifies the event category, e.g., LOG (log message), TRACE (call/return), STACK (stack trace), and IO (input/output). The field *attrs* is a structured attribute vector obtained by parsing and normalizing raw textual or monitoring records, including the log level and template

identifier, exception type, function summary, and resource usage. Finally, *ids* denotes the set of CI semantic entities involved in the event, such as a test case and its concrete execution instance (TestExec), threads, resource instances, environment configurations (EnvConfig), and code symbols (CodeSymbol). By uniformly parsing logs, call traces, stack traces, and monitoring data, rewriting timestamps, and pruning noisy fields, the UEM reduces heterogeneous records into this event space.

Building on the event space, we further map heterogeneous events to CI semantic entity types and construct an entity relation skeleton of the execution process. As illustrated in the middle and bottom layers of Figure 2, exception-related events (e.g., EXC) are abstracted as Exception entities; read/write and I/O events are mapped to Resource; environment-change events to EnvConfig; thread creation and scheduling events to Thread; method-call events to CodeSymbol; and assertion/test-outcome events are associated with TestCase and its concrete execution instance TestExec. The *ids* field in each event is used to instantiate CI entities and their semantic relations. In doing so, the UEM retains the essential information from raw runtime signals while exposing a well-defined set of node types and candidate relations for subsequent HEG construction.

Algorithm 1 presents the pseudocode of the UEM construction procedure developed in this study, which normalizes heterogeneous CI runtime artifacts into unified event tuples. The UEM construction process rewrites raw timestamps into a run-local logical time t^*. The happens-before (HB) relation is derived primarily from the total order of events within each thread, synchronization semantics such as locks and waits, and call–return matching constraints. When such synchronization evidence is missing, only conservative precedence relations are retained to avoid introducing spurious dependencies. The process further templates log messages, prunes noisy fields, and normalizes parsable signals into structured attributes, *attrs*. Entity identifiers, *ids*, are canonicalized in a unified namespace of a project–module–test method–run ID, ensuring that the same entity can be consistently aligned across runs and versions.

Algorithm 1 UEM construction and temporal alignment for a RunPair

Input : A target test case t and its contrastive run pair $\mathcal{R}_t = \{r^{\text{pass}}, r^{\text{fail}}\}$; artifacts indexed by run IDs; parsers/template extractors $\mathcal{P}$.
Output: Unified event set $\mathcal{E}_t = \{e_i\}$, where $e = \langle t^*, type, attrs, ids \rangle$; CI entity $\mathcal{V}_t$; happens-before constraints $\mathcal{H}_t$.

$\mathcal{E}_t \leftarrow \emptyset$; $\mathcal{V}_t \leftarrow \emptyset$; $\mathcal{H}_t \leftarrow \emptyset$
foreach $r \in \mathcal{R}_t$ **do**
  $\mathcal{M}_r \leftarrow \texttt{ParseLogs}(r, \mathcal{P}) \cup \texttt{ParseFailingReports}(r, \mathcal{P})$
  foreach $m \in \mathcal{M}_r$ **do**
    $type \leftarrow m.type$ `// e.g.,LOG, TRACE, STACK, ASSERT`
    $attrs \leftarrow \texttt{NormalizeAttrs}(m)$ $ids \leftarrow \texttt{ExtractEntities}(m, t, r)$ `// e.g., TestExec, Thread, Exception, Resource`
    $t^* \leftarrow \texttt{RewriteTimestamp}(m.timestamp)$ `// logical timestamp for within-run ordering`
    $\mathcal{V}_t \leftarrow \mathcal{V}_t \cup ids$ $\mathcal{E}_t \leftarrow \mathcal{E}_t \cup \{\langle t^*, type, attrs, ids \rangle\}$
  end
end
Sort $\mathcal{E}_t$ by t^* to obtain a total order within each run $\mathcal{H}_t \leftarrow \texttt{InferHB}(\mathcal{E}_t)$
return $\mathcal{E}_t, \mathcal{V}_t, \mathcal{H}_t$

Example 1. *Consider the following failing log message produced by the test framework:*

```
2025-06-03T10:15:12.345Z [TestRunner] AssertionError: expected 200
but got 500 at com.example.ApiTest#testGet()
```

After timestamp correction and template matching, this record is converted into an event

$$e = \langle t^*, ASSERT, attrs = \{templateId = \#A12,\ expect = 200,\ actual = 500\},$$
$$ids = \{TestCase = \texttt{ApiTest\#testGet@commitSHA}\}\rangle.$$

The UEM is constructed using deterministic normalization rules because it is intended to provide an interpretable and auditable evidence schema, rather than a black-box feature representation. CI logs and runtime artifacts are highly project-specific, and fully learned parsers may require additional labeled data or overfit to project-specific formats. Therefore, UEM first preserves explicit event types, timestamps, entities, and relations in a stable structured form, while task-specific embeddings are learned later by the heterogeneous graph encoder.

3.3. Heterogeneous Execution Graph and Multi-View Construction

Building on the UEM events and CI semantic entities defined in the previous section, we model a single test execution as a HEG. Formally, for a test case t and its concrete execution instance $\mathsf{TestExec}_t$ in a CI run, we construct a graph $G_t = (V_t, E_t, \phi_V, \phi_E)$. Here, V_t denotes the node set, consisting of CI semantic entities related to this execution. The edge set E_t contains directed edges that encode structural relations among these entities, derived from UEM events and their associated *ids*. The mappings $\phi_V : V_t \to \mathcal{T}_V$ and $\phi_E : E_t \to \mathcal{T}_E$ are the node-type and edge-type functions, respectively, distinguishing semantic roles and relation types, as illustrated in Figure 3.

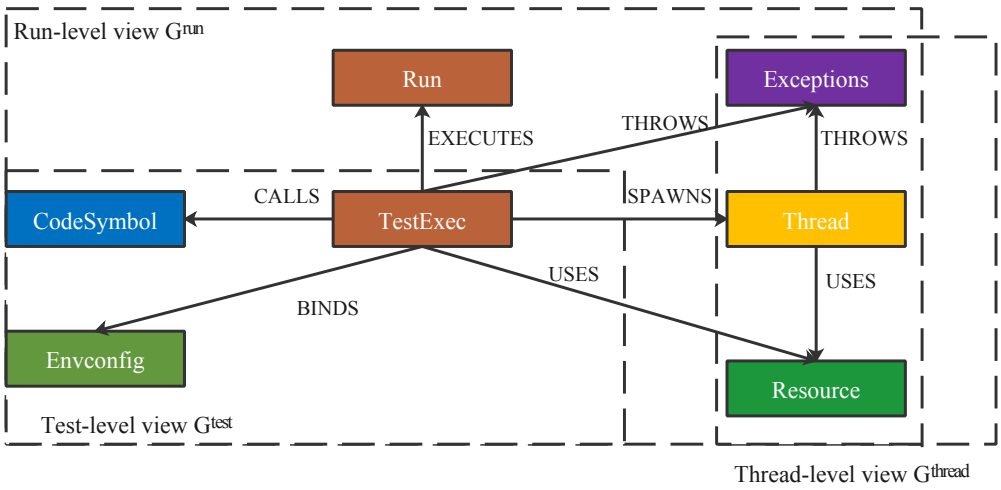


Figure 3. Example of the HEG and the three derived views for a single test execution.

At the node level, the HEG schema is designed according to three principles: extractability from UEM events, relevance to common flaky-test mechanisms, and cross-project stability. Specifically, the node types in graph G_t include a pipeline run Run and its pairwise relation RunPair, a test case TestCase and its concrete execution instance TestExec, thread entities Thread, resource entities Resource, exception entities Exception, code symbols CodeSymbol, and environment configuration entities EnvConfig. These entities can be consistently extracted from CI runtime artifacts and are closely related to common flaky-test causes. Correspondingly, relation types are defined to capture execution, invocation, thread creation, exception propagation, resource usage, and configuration binding, such as EXECUTES, CALLS, SPAWNS, THROWS, USES, and BINDS.

For feature initialization, an initial feature vector is constructed for each node based on the defined CI semantic entities. For TestCase and CodeSymbol nodes, tokens are extracted

from identifiers such as test class names, method names, package names, and method signatures, and static semantic representations are obtained using a pre-trained model such as CodeBERT [46]. On top of these static features, dynamic statistical features are aggregated from UEM events for each node type and then normalized. Finally, the initial feature vector of each node $v \in V_t$ is formed by concatenating the identifier embedding, statistical features, and node-type embedding, denoted as $\mathbf{x}_v \in \mathbb{R}^d$, which serves as the input representation for relation modeling in the subsequent heterogeneous graph Transformer.

At the edge level, multiple types of directed edges in G_t are introduced according to relations induced by UEM events, capturing structural dependencies among entities during test execution. Specifically, we add EXECUTES edges from a pipeline run to its internal test executions; BINDS, USES, and THROWS/RAISES edges from a test execution to its associated environment configurations, resources, and exceptions; and CALLS edges from a test execution to the invoked CodeSymbol nodes. In addition, we connect the Run node to the two corresponding Run nodes via a typed pairing relation to explicitly encode the contrast between passing and failing runs of the same test. Each edge $e = (u, v) \in E_t$ is associated with a type label $\phi_E(e)$, which is represented by a relation-type embedding in the model input.

Algorithm 2 presents the pseudocode of the HEG construction procedure developed in this study, which maps UEM events into typed runtime entities and typed relations. The node and edge types in the HEG are determined by the UEM-to-entity mapping rules, where each directed edge follows the semantic convention of "*actor entity* $\rightarrow$ *affected entity*", which supports inspecting influence paths in the resulting execution graph. For each Run of the same test, we explicitly connect the passing and failing runs, and can optionally attach run-level discrepancy attributes as edge features to strengthen the contrastive signal. The three views are sliced independently from the same HEG: the run-level view retains the global context associated with the target TestExec, the test-level view extracts a k-hop ego-subgraph centered at TestExec/TestCase, and the thread-level view focuses on long-lived threads and their interactions with resources and exceptions.

From the complete HEG G_t, multiple structural views are derived to capture flakiness patterns at different granularities. The three views are selected according to the main evidence granularity of flaky-test behavior. The run-level view captures global execution context, the test-level view focuses on local failure-related evidence around the target test, and the thread-level view models concurrency and resource-interaction patterns. The run-level view G_t^{run} retains the entities and relations associated with the target TestExec, characterizing its global context within a CI run. The test-level view G_t^{test} is centered at TestExec/TestCase and extracts a k-hop ego-subgraph, emphasizing the test case and its most relevant threads, resources, and exceptions. The thread-level view G_t^{thread} focuses on long-lived threads and the resources they interact with, highlighting concurrency scheduling and resource-contention patterns. Formally, the multi-view construction is denoted as $G_t^{(v)} = \psi_v(G_t)$, where $v \in \{\text{run}, \text{test}, \text{thread}\}$ indexes the view type. In the subsequent heterogeneous graph Transformer, HgtFlaky jointly encodes the three views and fuses their representations to obtain a more comprehensive test-level representation for flakiness prediction.

Algorithm 2 HEG construction and multi-view slicing

Input : Target test case t; UEM event set $\mathcal{E}_t$; entity registry $\mathcal{I}_t$; HB constraints $\mathcal{H}_t$; node types Φ_V and edge types Φ_E; view constructors $\psi_{\text{run}}, \psi_{\text{test}}, \psi_{\text{thread}}$ (implemented by slicing rules).

Output: HEG $G_t = (V_t, E_t, \phi_V, \phi_E, X)$ and three views $G_t^{\text{run}}, G_t^{\text{test}}, G_t^{\text{thread}}$.

```
// (1) Initialize typed nodes and node features
```
$V_t \leftarrow \emptyset$; $E_t \leftarrow \emptyset$ **foreach** $id \in \mathcal{I}_t$ **do**
  create node v_{id} with type $\phi_V(v_{id}) \in \Phi_V$
  $X[v_{id}] \leftarrow$ `InitNodeFeatures`$(v_{id}, \mathcal{E}_t)$
  $V_t \leftarrow V_t \cup \{v_{id}\}$
end
```
// (2) Add typed edges induced by UEM events via a mapping function
```
foreach $e = \langle t^*, type, attrs, ids \rangle \in \mathcal{E}_t$ **do**
  $\mathcal{R}_e \leftarrow \rho(e)$ `// map an event to one or more typed relations`
  foreach $(u, r, v, \mathbf{z}) \in \mathcal{R}_e$ **do**
    add directed edge $(u \to v)$ with $\phi_E(u \to v) = r$
    $E_t \leftarrow E_t \cup \{(u \to v)\}$
  end
end
```
// (3) Add temporal or causal edges from HB constraints
```
foreach $(u \prec v) \in \mathcal{H}_t$ **do**
  add directed edge $(u \to v)$ with $\phi_E(u \to v) = \mathsf{HB}$
  $E_t \leftarrow E_t \cup \{(u \to v)\}$
end
```
// (4) Add RunPair linkage (pass vs.  fail) if applicable
```
add node RunPair and connect it to the two Run nodes with typed edges
Define $G_t \leftarrow (V_t, E_t, \phi_V, \phi_E, X)$
```
// (5) Multi-view slicing from the same HEG
```
$G_t^{\text{run}} \leftarrow$ `RunSlice`(G_t, t)
$G_t^{\text{test}} \leftarrow$ `KhopSlice`$(G_t, \{\mathsf{TestExec/TestCase}\}, k, \Phi_E^{\text{test}})$ `// k-hop subgraph`
$G_t^{\text{thread}} \leftarrow$ `ThreadSlice`$(G_t, \{\mathsf{Thread}\}, \Phi_E^{\text{thread}})$
return $G_t, G_t^{\text{run}}, G_t^{\text{test}}, G_t^{\text{thread}}$

3.4. Heterogeneous Graph Transformer with Multi-View Representation Learning

In HgtFlaky, HEGs contain multiple node types and relation types, as well as dense connections induced by temporal and causal ordering. This motivates a relation-aware encoder that can distinguish heterogeneous runtime interactions and aggregate information over long-range dependencies. The contribution here is not to introduce a new Transformer architecture from scratch, but to adapt heterogeneous graph representation learning to task-specific runtime-evidence graphs constructed from CI artifacts. Specifically, HgtFlaky first derives run-level, test-level, and thread-level views from the UEM-based HEG, and then applies a shared heterogeneous graph Transformer to encode these views for flaky prediction. The overall architecture is shown in Figure 4.

For each target test case t, HgtFlaky derives three views from the UEM-based HEG: a run-level view, a test-level view, and a thread-level view, denoted by $G_t^{\text{run}}, G_t^{\text{test}}, G_t^{\text{thread}}$. These views capture the global CI context, the test-centric local neighborhood, and resource-interaction behaviors, respectively. The three views are then fed into a shared-parameter heterogeneous graph Transformer encoder. Given node semantic features, relation-type information, as well as structural positional encodings, the encoder produces a view-level graph representation $\mathbf{h}_t^{\text{run}}, \mathbf{h}_t^{\text{test}}, \mathbf{h}_t^{\text{thread}}$. A multi-view fusion module then aggregates these embeddings into a single test-level representation $\mathbf{h}_t$, which is passed to a prediction head to output the probability $\hat{y}_t$ that test t is flaky.

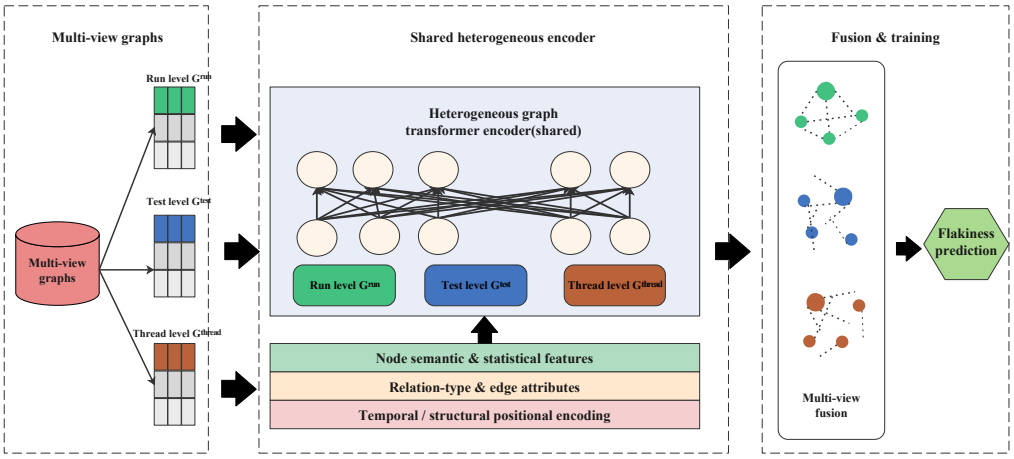


Figure 4. Overall architecture of the proposed multi-view heterogeneous graph Transformer for flaky test detection.

Relation-aware heterogeneous graph Transformer. The purpose of the heterogeneous graph Transformer is to aggregate information from different types of runtime entities while preserving relation-specific semantics. In the HEG, a dependency between a test execution and an exception is different from a dependency between a thread and a resource. Therefore, the encoder uses relation-specific projections and attention weights to distinguish different edge types during message passing.

For encoding a single view, we adopt a relation-aware heterogeneous graph Transformer to jointly model multi-typed nodes and multi-typed edges in the HEG. Given a view graph $G_t^{(v)} = (V_t^{(v)}, E_t^{(v)})$, each node $u \in V_t^{(v)}$ is initialized as $\mathbf{h}_u^{(0)} = \mathbf{x}_u$. Each edge $(u, w) \in E_t^{(v)}$ is associated with a relation type $r = \phi_E(u, w) \in \mathcal{R}$ and carry edge attributes $\mathbf{z}_{u,w}$. At layer l, learnable projections and relation-specific parameters are introduced to map heterogeneous neighborhoods into a unified attention space:

$$\mathbf{q}_u^{(l)} = \mathbf{W}_Q^{\tau(u)} \mathbf{h}_u^{(l-1)}, \quad \mathbf{k}_w^{(l,r)} = \mathbf{W}_K^r \mathbf{h}_w^{(l-1)}, \quad \mathbf{v}_w^{(l,r)} = \mathbf{W}_V^r \mathbf{h}_w^{(l-1)}, \tag{1}$$

where $\tau(u)$ denotes the node type. We then compute relation-aware attention scores and normalize them into weights. When edge attributes are available, we inject them via a learnable term:

$$e_{u,w}^{(l,r)} = \frac{\left(\mathbf{q}_u^{(l)}\right)^\top \mathbf{k}_w^{(l,r)}}{\sqrt{d}} + \left(\mathbf{a}_r\right)^\top \mathbf{z}_{u,w}, \qquad \alpha_{u,w}^{(l,r)} = \frac{\exp\left(e_{u,w}^{(l,r)}\right)}{\sum_{w' \in \mathcal{N}(u)} \exp\left(e_{u,w'}^{(l,\phi_E(u,w'))}\right)}. \tag{2}$$

Based on these weights, we perform weighted aggregation over each relation-specific neighborhood $\mathcal{N}_r(u)$ and sum across relations to obtain the aggregated update. The node update equation aggregates messages from relation-specific neighborhoods. Each incoming message is transformed according to its relation type before being weighted by the learned attention coefficient. This design enables HgtFlaky to preserve the semantics of heterogeneous runtime dependencies.

$$\mathbf{h}_u^{(l)} = \sum_{r \in \mathcal{R}} \sum_{w \in \mathcal{N}_r(u)} \alpha_{u,w}^{(l,r)} \, \mathbf{v}_w^{(l,r)}. \tag{3}$$

View-level representations and fusion. HgtFlaky adopts multi-head attention and averages the outputs of different heads to enhance the expressiveness of diverse relation interaction patterns. After L relation-aware layers, the model obtains contextualized node

representations $\mathbf{h}_u^{(L)}$, which capture key patterns of each view from structural, semantic, and behavioral perspectives. It then computes graph-level representations for test-level prediction. For each view $G_t^{(v)}$, an attention pooling function aggregates node representations into a view-level vector $\mathbf{h}_t^{(v)}$.

$$s_u^{(v)} = \mathbf{w}_p^\top \tanh(\mathbf{W}_p \mathbf{h}_u^{(L)}), \quad \beta_u^{(v)} = \frac{\exp(s_u^{(v)})}{\sum_{u' \in V_t^{(v)}} \exp(s_{u'}^{(v)})}, \quad \mathbf{h}_t^{(v)} = \sum_{u \in V_t^{(v)}} \beta_u^{(v)} \, \mathbf{h}_u^{(L)}. \tag{4}$$

Given the three view embeddings $\mathbf{h}_t^{\text{run}}, \mathbf{h}_t^{\text{test}}, \mathbf{h}_t^{\text{thread}}$, we construct a multi-view fusion module to produce a single test-level representation $\mathbf{h}_t$. A simple and effective implementation concatenates the three view vectors and compresses them via a multi-layer perceptron (MLP):

$$\mathbf{h}_t = \text{MLP}\Big([\mathbf{h}_t^{\text{run}} \parallel \mathbf{h}_t^{\text{test}} \parallel \mathbf{h}_t^{\text{thread}}]\Big). \tag{5}$$

The MLP-based fusion layer is used as a simple, stable, and parameter-efficient integration module. It combines the three view-level representations after relation-aware graph encoding without introducing excessive architectural complexity. No manually predefined weights are assigned to the three views. Their relative contributions are implicitly learned through the trainable parameters of the MLP under the joint objective. Finally, we feed $\mathbf{h}_t$ into a classification head $\text{CLS}(\cdot)$ to produce the probability $\hat{y}_t$ that the test is flaky:

$$\hat{y}_t = \sigma(\text{CLS}(\mathbf{h}_t)). \tag{6}$$

Learning objectives. For the learning objectives, we aim for the model to not only accurately distinguish flaky from non-flaky tests, but also learn transferable representations in CI settings where noise is prevalent and the data distribution may shift substantially across versions or projects. To this end, we adopt a joint optimization strategy that combines supervised classification with a multi-view consistency regularization.

Given the label of a test case t, denoted by $y_t \in \{0, 1\}$ (where 1 indicates a flaky test), the classification head outputs a predicted probability $\hat{y}_t$. We use the binary cross-entropy loss as the supervised objective:

$$\mathcal{L}_{\text{CE}} = -\sum_{t \in \mathcal{T}_{\text{train}}} \Big(y_t \log \hat{y}_t + (1 - y_t) \log(1 - \hat{y}_t) \Big). \tag{7}$$

Relying solely on supervised signals may cause the model to overfit to superficial patterns of a particular view or a specific project. Therefore, we introduce a multi-view consistency loss $\mathcal{L}_{\text{MV}}$, encouraging representations of the same test to be consistent across different views, while being distinguishable from representations of other tests. Concretely, embeddings of the same test under different views are treated as positive pairs, whereas embeddings from different tests form negative pairs. We adopt an InfoNCE-style contrastive objective and, for each anchor embedding $\mathbf{h}_t^{(v)}$, minimize to get

$$\mathcal{L}_{\text{MV}} = -\sum_{t \in \mathcal{T}_{\text{train}}} \sum_{v \neq v'} \log \frac{\exp\left(\text{sim}(\mathbf{h}_t^{(v)}, \mathbf{h}_t^{(v')})/\tau\right)}{\sum_{t'} \exp\left(\text{sim}(\mathbf{h}_t^{(v)}, \mathbf{h}_{t'}^{(v')})/\tau\right)}, \tag{8}$$

Finally, the overall training objective is defined as

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \, \mathcal{L}_{\text{MV}}, \tag{9}$$

where λ controls the strength of the consistency regularization. With this joint objective, the model leverages label supervision to improve discriminative performance while learning more robust representations from multi-view structures, thereby improving flaky-test detection under cross-version and cross-project settings.

4. Validation

4.1. Research Questions

RQ1: How effective is the proposed approach for flaky-test detection?

The primary goal of flaky-test detection is to accurately distinguish flaky tests from stable ones in realistic CI environments. This research question first investigates the overall detection effectiveness of the proposed approach on a standard flaky-test dataset. This analysis aims to validate the effectiveness of modeling flaky behaviors through multi-view HEGs and employing a Transformer-based encoding framework for flaky test detection.

RQ2: How does HgtFlaky compare to state-of-the-art flaky-test detectors?

Beyond establishing the overall effectiveness of HgtFlaky, it is essential to understand how it performs relative to representative state-of-the-art (SOTA) baselines that follow fundamentally different design paradigms. In particular, prior work has shown competitive results using feature-based supervised learning (e.g., FlakeFlagger [20]) and language-model-based prediction that relies solely on test source code (e.g., Flakify [21]). This research question therefore conducts a comparison between HgtFlaky and these SOTA detectors under the same dataset and evaluation metrics, so that any performance differences can be attributed to modeling choices rather than experimental confounders.

RQ3: Does multi-view HEG modeling significantly improve flaky-test detection performance?

A key design of the proposed method lies in characterizing flaky behaviors from three complementary views, which are jointly modeled by a shared heterogeneous graph Transformer. However, multi-view modeling inevitably introduces additional structural complexity and computational overhead, making it necessary to examine its actual contribution to performance. Through ablation studies, this research question analyzes the impact of individual views and their combinations on detection performance, investigates the role of each view in capturing flaky behavior patterns, and examines whether the multi-view fusion mechanism consistently yields stable and meaningful performance gains.

4.2. Dataset and Benchmarks

To evaluate the proposed approach, two publicly available flaky-test datasets are used. The first dataset is the FlakeFlagger dataset [20]. The second dataset is the International Dataset of Flaky Tests (IDoFT), which comprises many datasets for flaky-test cases used by previous studies on flaky-test case prediction [4,47,48].

FlakeFlagger dataset.

We use the FlakeFlagger dataset released by Alshammari et al. [20] as the primary benchmark. Table 1 summarizes the FlakeFlagger artifacts and their mappings to UEM events and HEG components. It contains flakiness information for 22,236 Java test methods drawn from 23 open-source GitHub projects. The subject systems vary substantially in test-suite scale, ranging from 55 to 6267 tests per project, enabling evaluation across both small and large CI workloads. All projects are implemented in Java and built with Maven, and each instance corresponds to a single JUnit-style test method. Besides the test source code, the dataset also includes the engineered features originally used by FlakeFlagger, which facilitates direct comparison with prior feature-based baselines. Each test is labeled as flaky or non-flaky via extensive reruns: the label is derived from executing the test

10,000 times and observing outcome variability. We further leverage the accompanying rerun artifacts to reconstruct run-level evidence for selected passing–failing run pairs. We parse these artifacts into UEM events and map them into HEGs, yielding structured multi-view inputs beyond code-only signals.

Table 1. FlakeFlagger artifacts and extracted evidence for constructing UEM events and HEGs.

Artifact	Extracted Fields (Examples)	Mapped UEM/HEG Components
`test_results.csv`	test identifier, run id, outcome (pass/fail), duration, rerun counts	labels y_t; RunPair sampling (r^{pass}, r^{fail}); nodes: Run, TestCase, TestExec; edges: EXECUTES, INSTANCE_OF, RunPair links
`build-logs*.tgz`	log level, thread name, message template, timestamps (if available), test start/end markers, framework signals	UEM events: LOG, THREAD_DUMP; inferred ordering t^* and happens-before; nodes: Thread; edges: HB/BEFORE (within-run)
`failing-test-reports*.tgz` (Surefire XML)	failure message, exception type, stack trace frames, failing assertion site, JVM/OS properties	UEM events: EXC, STACK, ASSERT, TESTCASE_FAIL; nodes: Exception, CodeSymbol, EnvConfig; edges: THROWS, RAISES, CALLS, STACK_AT, BINDS

Note: The asterisk (∗) denotes a wildcard matching one or more artifact archive files.

IDoFT dataset.

IDoFT is a large-scale collection of flaky tests mined from real-world Java projects. The original release contains 3742 flaky-test cases spanning 314 GitHub repositories. Notably, flakiness in IDoFT may be established under different runtime environments and with varying rerun budgets, reflecting heterogeneous collection settings. In practice, a subset of instances cannot be retrieved or reproduced because the referenced repositories and commits are no longer accessible. Following the curation protocol commonly adopted in prior work, we discard 474 test cases whose test code cannot be recovered, resulting in 3268 flaky tests from 312 projects.

4.3. Experiment Design

4.3.1. Baseline and Evaluation Metrics

To systematically assess the effectiveness and practicality of HgtFlaky, we compare against two representative baselines that instantiate distinct modeling paradigms: FlakeFlagger, a feature-engineering-driven learning framework, and Flakify, a language-model-based black-box predictor. To ensure fairness and reproducibility, all methods are trained and evaluated on the same benchmark under an identical data-splitting protocol, for the same prediction task, and are compared using the same evaluation metrics. For per-project validation, the data are split at the project level. In each fold, all test cases and run pairs from one target project are held out for testing, while the remaining projects are used for training and validation. To reduce leakage risk, project-specific identifiers are excluded from the model input. Duplicate test identifiers and RunPair records are also checked to ensure that the same test instance does not appear in both training and testing sets.

For the FlakeFlagger baseline, we train the model using the complete feature set released with the dataset, without adding new features or removing existing ones. The feature set combines black-box signals with white-box signals. This enables FlakeFlagger to estimate flakiness risk from engineered measurements without explicitly leveraging multi-source runtime evidence. Following the original setting, we train a supervised classifier

directly on the released feature files and labels, and evaluate it under the same data-splitting protocol as HgtFlaky.

Flakify takes the test source code as the sole input and leverages a language model to infer code semantics and regularities, thereby predicting whether a test is flaky. It does not rely on additional evidence such as coverage information, change history, or runtime logs, and thus represents a code-text-only paradigm for flaky-test prediction. In experiments, we follow the official implementation of Flakify and its default hyper-parameter settings, and train/evaluate it on the same benchmark under the same data-splitting protocol as HgtFlaky.

Regarding the evaluation protocol, we adopt the cross validation and per-project validation setting that has been commonly used in prior studies on the FlakeFlagger dataset, in order to better reflect practical deployment scenarios. For per-project validation, in each round we hold out all test cases from one project as the test set and train the model on the remaining projects. This procedure is repeated for every project in the dataset, and performance is summarized at the project level. We report Precision, Recall, and F1-score for the flaky/non-flaky binary classification task, and use the distribution of per-project results for statistical analysis. To assess statistical significance, we further apply Fisher's exact test to compare the proportions of correctly classified instances between methods on each held-out project. This allows us to determine whether HgtFlaky yields statistically significant improvements over the baselines under cross-project transfer.

4.3.2. Training and Testing Prediction Models

Figure 5 illustrates how we transform the rerun artifacts released with the FlakeFlagger dataset into structured inputs for graph representation learning. We first obtain the execution outcomes and run identifiers of each test case from `test_results.csv`, and then sample a contrastive RunPair for the same test to characterize outcome variability under identical code and configuration. Next, we parse the corresponding `build-logs.tgz` and `failing-test-reports.tgz` artifacts indexed by these run IDs, extracting multi-source runtime evidence including log messages, thread information, exception types and stack traces, assertion failures, and environment properties. Each raw record is normalized into a UEM event tuple $e = \langle t^*, type, attrs, ids \rangle$, enabling cross-source temporal alignment and semantic unification. Based on the resulting events, we construct a HEG for each target test. CI semantic entities are modeled as typed nodes, and relations are encoded as typed directed edges. Finally, to capture flakiness patterns at different granularities, we slice each HEG into three view-specific subgraph views denoted by $G_t^{\text{run}}, G_t^{\text{test}}, G_t^{\text{thread}}$, which serve as the multi-view inputs to HgtFlaky.

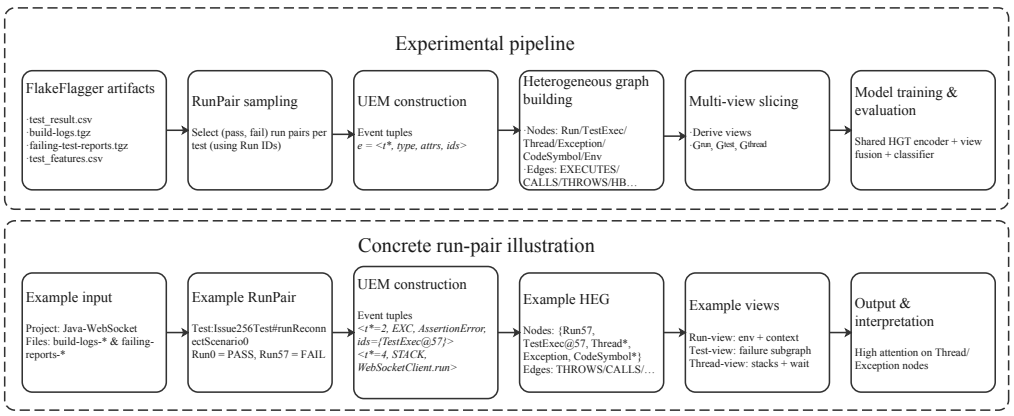


Figure 5. Experimental pipeline for transforming FlakeFlagger artifacts into UEM events, HEGs, and multi-view graph inputs. Asterisks indicate one or more matching artifact files or entity instances.

Ten-fold Cross-Validation. We first evaluate all models using a 10-fold stratified cross-validation protocol to preserve the class distribution across splits and obtain a reliable estimate of overall performance. Concretely, we partition the dataset at the test-case level into 10 stratified folds. In each fold, 90% of the test cases are used for training and the remaining 10% for testing. Since flaky tests typically form a minority class, we handle class imbalance on the training and validation sets, while keeping the test set in its original distribution to avoid overestimating performance under realistic CI conditions. For HgtFlaky, graph construction is performed after the train/test split in each fold and independently per test case, ensuring that no information from the test fold is used during training. We report the mean performance over the 10 folds.

Per-Project Validation. To more realistically assess predictive performance on unseen projects, we additionally adopt per-project validation for cross-project evaluation. Specifically, for each project P in the dataset, we hold out all test cases from P as the test set and train the model on test cases from all remaining projects. A portion of the training data from the non-held-out projects only is further reserved as a validation set for early stopping and hyper-parameter tuning. This procedure is repeated for every project, such that each project serves once as a completely unseen target domain. Consistent with the cross-validation setting, we do not perform resampling or class balancing on the test set, so that the evaluation distribution remains reflective of practical CI settings. For per-project validation, one project is held out as the target test project in each fold. The remaining projects are used as source projects. To tune hyper-parameters without leaking information from the target project, a separate validation subset is split only from the source projects. After the hyper-parameters are selected on the validation subset, the final model is evaluated on the held-out target project. The final hyper-parameter setting of HgtFlaky is as follows. The hidden dimension is set to 128, the heterogeneous graph Transformer contains two layers, and each layer uses four attention heads. The dropout rate is set to 0.3. The model is trained with a learning rate of 5×10^{-4}, a weight decay of 1×10^{-4}, and a batch size of 32.

4.4. Results

4.4.1. RQ1 Results

Table 2 summarizes the overall detection performance of HgtFlaky on the FlakeFlagger and IDoFT datasets under 10-fold cross-validation. The results are reported as mean $\pm$ standard deviation across the 10 folds. Overall, HgtFlaky achieves strong and relatively stable performance on both datasets. On FlakeFlagger, it obtains a Precision of $74.0 \pm 2.8\%$, a Recall of $93.0 \pm 1.9\%$, and an F1-score of $83.0 \pm 2.2\%$. On IDoFT, it achieves near-ceiling performance, with $99.0 \pm 0.6\%$ Precision, $98.0 \pm 0.8\%$ Recall, and $98.0 \pm 0.7\%$ F1-score. The relatively small standard deviations indicate that the model maintains stable performance across different folds.

Table 2. Overall cross-validation performance of HgtFlaky on two datasets.

Approach	Dataset	Precision (%)	Recall (%)	F1-Score (%)
HgtFlaky	FlakeFlagger	74.0 ± 2.8	93.0 ± 1.9	83.0 ± 2.2
HgtFlaky	IDoFT	99.0 ± 0.6	98.0 ± 0.8	98.0 ± 0.7

On FlakeFlagger, HgtFlaky exhibits a high-recall but relatively lower-precision profile, suggesting that it can recover most flaky tests while still misclassifying a non-trivial portion of stable tests as flaky. This indicates that FlakeFlagger contains more heterogeneous runtime behaviors and noisier decision boundaries, where reducing false positives remains challenging. In contrast, the near-ceiling and low-variance results on IDoFT should be interpreted with caution. Similar high performance has also been reported by Flakify

on IDoFT, suggesting that this dataset may contain more consistent or separable signals for learning-based flaky-test prediction [21]. Therefore, the IDoFT results are used as complementary evidence of the effectiveness of HgtFlaky, rather than as an indication that flaky-test detection has been fully solved.

Table 3 reports descriptive statistics of the per-project evaluation results, shedding light on the cross-project stability of HgtFlaky. On FlakeFlagger, project-level performance exhibits more pronounced variability. The minimum Precision drops to 21%, whereas the median reaches 81% and the upper quartile reaches 95%. For Recall, the mean and median are 89% and 94%, respectively, with the upper quartile reaching 100%. Overall, these results suggest that HgtFlaky can consistently recover flaky tests on the majority of projects, while a small number of projects form a long tail with substantially lower Precision or Recall. Such cases may stem from project-specific testing practices, dependency stacks, and environment-related noise, which can shift the distribution of observable runtime evidence. In comparison, IDoFT shows a more concentrated cross-project distribution: Precision achieves a mean/median of 95%/100%, and F1 achieves a mean/median of 93%/100%, indicating that most projects reach very strong performance. Nevertheless, a few projects still suffer from degraded Recall (minimum 27%), suggesting that even in an overall easier benchmark, certain projects may exhibit noisier or more diverse flakiness mechanisms that remain challenging under cross-project transfer.

Table 3. Descriptive statistics of per-project validation results for HgtFlaky.

Dataset	Metric	Min (%)	25% (%)	Mean (%)	Median (%)	75% (%)	Max (%)
FlakeFlagger	Precision	21	67	75	81	95	100
	Recall	12	78	89	94	100	100
	F1-score	17	69	81	85	97	100
IDoFT	Precision	76	87	95	100	100	100
	Recall	27	84	92	100	100	100
	F1-score	41	85	93	100	100	100

4.4.2. RQ2 Results

Under the per-project validation setting, we compared HgtFlaky against two representative baselines, Flakify and FlakeFlagger. As shown in Table 4, HgtFlaky achieves the best cross-project generalization performance. On the FlakeFlagger dataset, it attains an average Precision/Recall/F1 of **78%/89%/81%**, consistently outperforming Flakify (72%/85%/73%) across all three metrics. In contrast, FlakeFlagger performs substantially worse under this cross-project protocol, reaching only 15%/14%/7%. These results indicate that, compared to solutions relying solely on test-code text or pre-computed metrics, HgtFlaky benefits from explicitly leveraging runtime evidence and multi-relational structural information, thereby being more resilient to distribution shifts across projects and yielding more robust detection in the realistic new-project CI scenario.

At the project level, HgtFlaky maintains consistently strong performance on the majority of projects. For instance, it achieves an F1-score of 100% on achilles, alluxio, and ninja, and remains above 90% on hbase, hector, http-request, and wro4j, demonstrating strong cross-project transferability. While Flakify also performs well on many projects, it is more prone to degraded Precision or F1 on several projects, whereas HgtFlaky provides a stronger floor on these cases, achieving F1-scores of 57%, 67%, 67%, and 62%, respectively. Nevertheless, HgtFlaky still exhibits a small number of long-tail difficult projects, suggesting that some projects may involve more dispersed flaky behaviors, sparser evidence, or noisier signals, which can hinder the transfer of cross-project patterns.

Table 4. Per-project results (%) under per-project validation on the FlakeFlagger dataset. Best F1-score per project is highlighted in bold.

Project	HgtFlaky			Flakify			FlakeFlagger		
	P	**R**	**F1**	**P**	**R**	**F1**	**P**	**R**	**F1**
achilles	100	100	**100**	100	100	**100**	0	0	0
activiti	90	94	**92**	80	90	85	2	94	4
alluxio	100	100	**100**	99	100	99	100	13	24
ambari	80	94	**86**	75	95	84	39	61	47
assertj-core	40	100	**57**	25	100	40	0	0	0
commons-exec	50	100	**67**	25	100	40	0	0	0
elastic-job-lite	50	100	**67**	50	100	60	0	0	0
handlebars.java	45	100	**62**	30	100	50	0	0	0
hbase	85	100	**92**	79	98	88	72	33	45
hector	100	86	92	100	93	**96**	0	0	0
http-request	95	92	**93**	88	88	88	0	0	0
httpcore	80	90	**85**	74	90	81	7	4	5
incubator-dubbo	25	40	**31**	6	16	9	7	32	12
java-websocket	98	100	**99**	95	95	95	0	0	0
logback	88	85	**86**	85	81	83	0	0	0
ninja	100	100	**100**	100	100	**100**	0	0	0
okhttp	80	82	**81**	78	85	**81**	100	2	4
orbit	86	100	92	88	100	**93**	0	0	0
spring-boot	43	12	**19**	40	1	2	9	3	4
undertow	80	86	**83**	75	85	79	7	43	12
wildfly	70	95	**81**	65	91	76	6	26	10
wro4j	93	100	**96**	88	100	94	1	19	3
zxing	100	85	**92**	100	50	66	0	0	0
Overall	78	89	81	72	85	73	15	14	7

To further examine the generalization boundary of HgtFlaky, projects with degraded performance in Table 4 are analyzed. Two typical patterns can be observed. First, projects such as spring-boot and incubator-dubbo obtain relatively low F1-scores, suggesting that some flaky behaviors are highly project-specific and cannot be fully captured by evidence patterns learned from other projects. Second, projects such as assertj-core, commons-exec, and handlebars.java show high Recall but relatively low Precision. This indicates that HgtFlaky can identify most flaky tests, but may misclassify stable tests when their runtime evidence resembles flaky executions. These observations suggest that HgtFlaky generalizes better when flakiness is reflected in transferable runtime structures. Its performance may degrade under project-specific frameworks, sparse flaky samples, or incomplete runtime evidence.

The comparison among HgtFlaky, Flakify, and FlakeFlagger should be interpreted with respect to their different input modalities. FlakeFlagger mainly relies on pre-computed black-box and white-box features. While such features can be relatively consistent within a single project, they are susceptible to cross-project distribution shift. Differences in test organization, coverage toolchains, and dependency environments may alter both the distribution and the practical meaning of the same feature. Flakify learns directly from test source code and can capture certain code-level regularities, but it remains constrained by textual representations and thus cannot explicitly encode runtime flakiness mechanisms. By contrast, HgtFlaky aligns heterogeneous runtime evidence through UEM, maps it into a heterogeneous structure, and encodes multi-relational dependencies across multiple views. Therefore, the performance gain of HgtFlaky should be interpreted as the benefit of richer runtime-evidence modeling rather than a classifier-only advantage.

To further assess statistical significance, we apply a two-sided Fisher's exact test to compare the proportions of correctly classified test cases between methods. The results show that HgtFlaky correctly classifies a significantly larger proportion of test cases than

FlakeFlagger ($p < 0.05$), indicating that UEM-based multi-view heterogeneous execution evidence can provide a more stable discriminative signal under cross-project transfer. Moreover, HgtFlaky also yields a statistically significant advantage over Flakify, suggesting that black-box prediction based solely on test-code text may fail to capture key behavioral differences that drive flaky outcomes in some projects.

Overall, under per-project validation on the FlakeFlagger dataset, HgtFlaky achieves Precision 78%, Recall 89%, and F1-score 81%. Compared to Flakify (72%/85%/73%), HgtFlaky improves by approximately +5.3/+3.7/+8.0 percentage points on Precision/Recall/F1, respectively, while the margin over FlakeFlagger is substantially larger. These findings demonstrate that, in the more realistic project setting, HgtFlaky better preserves overall detection quality, achieving a stronger Precision–Recall balance as reflected by the higher F1-score.

4.4.3. RQ3 Results

The full multi-view HgtFlaky model is compared with its single-view variants through an ablation study under the per-project validation setting. As shown in Figure 6, the full model achieves Precision/Recall/F1-score of 78/89/81, outperforming all single-view counterparts. Specifically, the F1-scores of Test-only, Run-only, and Thread-only are 78, 74, and 67, respectively, indicating that evidence from a single granularity is insufficient to robustly cover the diverse manifestations of flaky behaviors. In terms of F1-score, the multi-view model improves over Test-only, Run-only, and Thread-only by $+3$, $+7$, and $+14$ percentage points, respectively, highlighting the benefit of fusing complementary cues across views while mitigating view-specific noise.

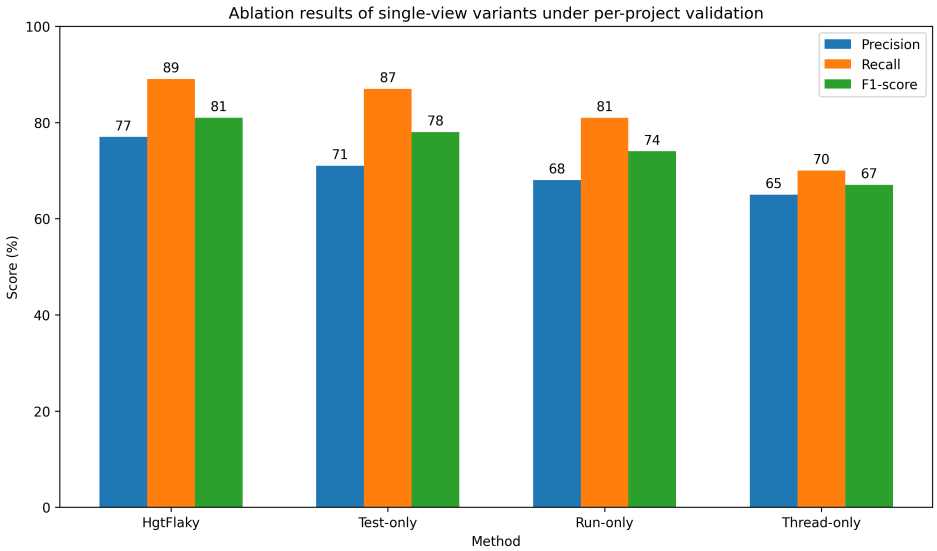


Figure 6. Ablation results of single-view variants under per-project validation.

The relative performance among single-view variants further reveals the contribution of each view. Test-only is the strongest single-view baseline, indicating that the local structure centered on test execution and failure-related evidence provides the most direct signals for identifying flakiness. Run-only ranks second. While it captures additional run-level contextual information, it also introduces weakly relevant noise, making it harder to preserve both Precision and Recall simultaneously. Thread-only performs the worst, implying that although concurrency and resource-interaction cues are closely tied to the root causes of flakiness. They often exhibit substantial cross-project variability when used in isolation, thus limiting the stability and transferability of learned patterns. Overall, multi-view fusion enables HgtFlaky to jointly exploit complementary information across

the three views within a unified framework, leading to more robust performance in cross-project evaluation.

As shown in Table 5, removing the multi-view consistency loss L_{MV} decreases the Precision/Recall/F1-score from 78%/89%/81% to 73%/86%/79%, respectively. This confirms that the contrastive consistency objective helps HgtFlaky learn more view-consistent and transferable representations under per-project validation. The larger decrease in Precision further suggests that L_{MV} mitigates view-specific noisy patterns and reduces false positives. Meanwhile, the ablated model still outperforms most single-view variants, indicating that the overall effectiveness mainly comes from multi-view heterogeneous evidence modeling and is further strengthened by consistency regularization.

Table 5. Ablation results of multi-view consistency under per-project validation.

Method	Precision (%)	Recall (%)	F1-Score (%)
HgtFlaky w/o L_{MV}	73	86	79
HgtFlaky	78	89	81

To examine whether the performance gain comes from relation-aware heterogeneous graph encoding rather than merely from using graph-structured inputs, several graph encoder variants are compared under the same UEM-based HEG construction and data-splitting protocol. Specifically, the heterogeneous graph Transformer is replaced with GCN, GAT, and R-GCN encoders, while the remaining components are kept unchanged.

As shown in Table 6, HgtFlaky achieves the best performance among all encoder variants, with 78% Precision, 89% Recall, and 81% F1-score. GCN obtains the lowest F1-score because it treats the execution graph as a relatively homogeneous structure and cannot explicitly distinguish different runtime relation types. GAT improves over GCN by introducing attention-based neighborhood aggregation, but it still lacks explicit relation-type modeling. R-GCN further improves the F1-score to 78%, confirming the usefulness of relation-aware message passing for HEGs. Compared with HGT without multi-view fusion, the full HgtFlaky model further improves the F1-score from 79% to 81%, indicating that multi-view evidence modeling provides complementary benefits beyond the graph encoder itself.

Table 6. Comparison of graph encoder variants under per-project validation.

Encoder	Precision (%)	Recall (%)	F1-Score (%)
GCN	66	80	72
GAT	70	82	75
R-GCN	73	85	78
HGT w/o multi-view fusion	74	86	79
HgtFlaky	78	89	81

Figure 7 shows the sensitivity of HgtFlaky to the regularization coefficient λ. The model achieves the best performance when $\lambda = 0.5$, reaching 78% Precision, 89% Recall, and 81% F1-score. When $\lambda = 0$, the model is trained without multi-view consistency regularization, and the F1-score drops to 79%, confirming the positive contribution of the contrastive consistency objective. When λ is further increased to 0.7 or 1.0, the performance slightly decreases, suggesting that excessive consistency regularization may suppress view-specific discriminative information. Based on these results, $\lambda = 0.5$ is used as the default setting in HgtFlaky. Overfitting is mitigated through project-level evaluation, source-project-only validation, dropout and weight decay. In per-project validation, the target project is used only for testing and is not involved in training and model selection.

However, projects with very few flaky instances may still introduce unstable decision boundaries, which remains a potential threat to generalization.

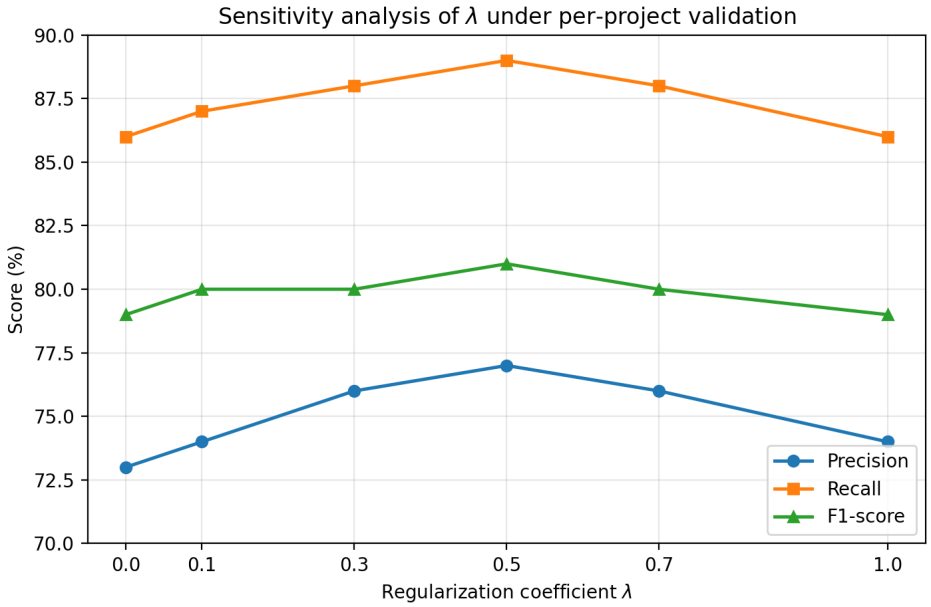


Figure 7. Sensitivity analysis of the regularization coefficient λ under per-project validation.

4.5. Discussion

Overall, the experimental results indicate that the advantage of HgtFlaky mainly comes from modeling flaky behavior as multi-view heterogeneous execution evidence, rather than relying only on pre-computed metrics or test-code representations. By jointly exploiting runtime artifacts, typed runtime entities, and multi-relational dependencies, HgtFlaky achieves more robust cross-project performance than existing baselines. The ablation results further show that the three views provide complementary evidence.

In addition to the quantitative per-project comparison in Table 4, the methodological differences among HgtFlaky, Flakify, and FlakeFlagger can be further summarized from several criteria. FlakeFlagger mainly relies on static and test-level features, and therefore provides limited support for modeling runtime structural dependencies. Flakify improves flaky-test prediction by using language-model-based representations, but its input is still mainly centered on code/textual evidence and does not explicitly construct heterogeneous runtime relations. In contrast, HgtFlaky uses multi-source runtime artifacts as the main evidence source. Therefore, compared with FlakeFlagger and Flakify, HgtFlaky provides stronger support for runtime structural modeling, heterogeneous relation learning, multi-granularity evidence integration, and evidence-aware interpretation. This methodological difference explains why HgtFlaky achieves more robust cross-project performance in Table 4.

Despite these improvements, the effectiveness of HgtFlaky still depends on the quality and completeness of CI runtime artifacts. Performance degradation on some projects may be caused by incomplete logs, noisy runtime records, project-specific dependencies, or heterogeneous execution environments. Therefore, beyond classification accuracy, it is necessary to examine the practical cost, scalability, and robustness of the proposed framework.

Table 7 reports the computational overhead of the major stages in HgtFlaky. The main preprocessing overhead comes from UEM construction, which parses runtime artifacts and normalizes them into event tuples. HEG construction and multi-view slicing introduce relatively small additional overhead, since each event is mapped to a limited number of typed entities and relations, and the three views are extracted from bounded subgraphs.

The inference cost of the heterogeneous graph Transformer remains low because attention is applied to compact view-level graphs. On FlakeFlagger, HgtFlaky requires about 36.8 s per training epoch and 3.9 GB peak GPU memory. On IDoFT, the corresponding cost is lower due to its smaller curated scale. These results suggest that HgtFlaky is practical for offline training and lightweight online prediction in CI environments. Nevertheless, for very large industrial pipelines with dense logs or long-running tests, incremental UEM construction, graph caching, and sampling-based view extraction may be needed to further improve scalability.

Table 7. Computational cost and scalability analysis of HgtFlaky. The runtime values are reported as average costs in our implementation.

Stage	Complexity	FlakeFlagger	IDoFT
UEM construction	$O(\|R\|)$	0.16 ± 0.04 s/test	0.11 ± 0.03 s/test
HEG construction	$O(\|V\| + \|E\|)$	0.05 ± 0.02 s/test	0.04 ± 0.01 s/test
Multi-view slicing	$O(\|V\| + \|E\|)$	0.03 ± 0.01 s/test	0.02 ± 0.01 s/test
HGT inference	$O(\sum_v \|E_v\| Hd)$	0.012 ± 0.004 s/test	0.010 ± 0.003 s/test
Training per epoch	–	36.8 ± 2.5 s	7.4 ± 0.8 s
Peak GPU memory	–	3.9 GB	1.6 GB

In addition to model runtime, the practical deployment cost of HgtFlaky mainly comes from collecting, parsing, and maintaining multi-source CI artifacts. Logs, resource events, and configuration information may require additional instrumentation or adaptation to different CI platforms. Changes in log formats, testing frameworks, or dependency environments may also require updates to the UEM parser. To reduce this cost, UEM is designed as a modular normalization layer, so that project-specific parsers can be updated without changing HEG construction and graph encoding.

The reliability of UEM-based graph construction is further analyzed under imperfect CI artifacts. Specifically, different components of the runtime evidence are perturbed to simulate common noise sources in CI environments. Event dropping simulates incomplete CI logs, attribute masking simulates parsing errors, and timestamp perturbation simulates unreliable temporal alignment. As shown in Table 8, HgtFlaky shows only moderate performance degradation under all three perturbation settings. When 10% of UEM events are randomly removed, the F1-score decreases slightly from 81% to 80%, indicating that the multi-view heterogeneous representation can tolerate a certain degree of missing runtime records. Attribute masking causes a larger decrease, reducing the F1-score to 79%, which suggests that event attributes provide useful semantic cues for flaky-test detection. Timestamp perturbation leads to the most pronounced degradation, with the F1-score dropping to 78%. This result indicates that temporal ordering and HB relations are important for capturing flaky behaviors. Overall, these results suggest that HgtFlaky can tolerate moderate noise in runtime evidence, but severe artifact incompleteness or systematic parsing errors may still affect the quality of UEM construction and the reliability of the resulting detection.

Table 8. Robustness analysis of HgtFlaky under incomplete runtime evidence.

Setting	Perturbed Component	Precision (%)	Recall (%)	F1-Score (%)
Original	None	78	89	81
Event dropping (10%)	UEM events	75	87	80
Attribute masking (10%)	Event attributes	74	86	79
Timestamp perturbation (10%)	Temporal order/HB edges	73	85	78

5. Threats to Validity

Construct Validity. Threats to construct validity mainly stem from whether the employed datasets sufficiently represent flakiness risks in real-world CI settings. First, the flaky labels in the FlakeFlagger and IDoFT datasets are derived from predefined execution strategies and labeling rules, which may not fully capture more complex triggering conditions in industrial CI environments. As a result, the labels can contain noise or bias. Second, we formulate the task as a binary classification problem and primarily use Precision, Recall, and F1-score for evaluation. While these metrics quantify predictive correctness, they do not directly measure practical governance benefits. Therefore, even if the model performs well on classification metrics, the extent to which it translates into cost savings in practice remains uncertain.

Internal Validity. Threats to internal validity relate to whether implementation details, data processing, and training procedures introduce confounding factors that affect the fairness of comparisons. First, different parser configurations or rule implementations may change the constructed heterogeneous graph topology and feature distributions, thereby affecting model performance. Moreover, inconsistencies across data splits or projects may introduce information leakage or bias. Second, training HgtFlaky depends on multiple hyper-parameters, and differences in hyper-parameter tuning effort relative to baselines could make part of the performance gap attributable to training resources rather than the proposed method itself. Third, reproducing baselines may also pose internal validity risks. For example, the language-model version and random seeds used by Flakify, as well as the classifier choice and feature preprocessing of FlakeFlagger, can influence the final results. To mitigate these threats, we follow publicly released implementations and default settings as closely as possible, and train and evaluate all methods under the same data-splitting protocols and evaluation metrics. We also report results under both cross-validation and per-project validation settings, and conduct ablation studies to verify the contribution of key components, strengthening the credibility of causal interpretation.

External Validity. Threats to external validity concern the generalizability of findings to other software ecosystems and CI platforms. First, the FlakeFlagger and IDoFT datasets mainly cover open-source Java projects with specific build and testing toolchains. Thus, our conclusions may not directly transfer to other languages or different build systems and test frameworks. Although project-level splitting reduces direct leakage between training and testing projects, it cannot completely eliminate shared runtime patterns caused by common third-party libraries or frameworks. Many Java projects depend on multiple overlapping libraries, making mutually exclusive library-based splits difficult to construct without producing very small or ill-defined folds. Therefore, this study reports project-level generalization and treats library-level generalization as a remaining threat to external validity and an important direction for future work. Second, HgtFlaky relies on the availability and completeness of CI artifacts. In some industrial CI environments, artifacts may be collected at insufficient granularity, which can degrade UEM construction quality and limit model applicability. Third, project scale and the distribution of flaky root causes can vary substantially across organizations, potentially introducing new flakiness triggers that require further adaptation. In future work, we will extend the evaluation to additional languages and toolchains, and assess robustness under varying levels of CI artifact availability to better validate applicability in broader engineering contexts.

6. Conclusions

This paper investigates flaky-test detection in CI settings and presents HgtFlaky, a multi-view heterogeneous graph learning framework centered on runtime evidence. A UEM is introduced to normalize heterogeneous CI artifacts into a semantically con-

sistent and temporally alignable event space. Built upon UEM, a HEG is constructed, and three complementary views at the run, test, and thread levels are derived to capture flakiness from different granularities. A heterogeneous graph Transformer is then employed to jointly encode the multi-view graph instances. The model is optimized with supervised classification and multi-view consistency regularization to improve robustness and transferability across projects. Future work will explore finer-grained event modeling and noise-robust learning, and extend the approach to additional programming languages and CI toolchains to further enhance applicability in real-world engineering environments.

Author Contributions: Conceptualization, P.D. and Y.G.; methodology, P.D. and X.M.; software, P.D. and X.M.; validation, X.M. and Y.Z.; investigation, P.D.; resources, Y.G.; writing—original draft preparation, P.D.; writing—review and editing, X.M.; supervision, Y.G.; project administration, X.M. and Y.Z.; funding acquisition, P.D. and X.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Excellent Scientific Research and Innovation Team of Anhui Colleges (Grant No. 2022AH010098), the Key Research Projects of Universities in Anhui Province (Grant No. 2024AH051366, No. 2024AH051368), the 2023 China University Industry University Research Innovation Fund-Digital Intelligence Science and Education Project (Grant No. 2023RY017), the Industry University Cooperative Education Project by the Ministry of Education (Grant No. 231107601104332), the Key Natural Science Project of the Education Department of Anhui Province (Grant No. 2025AHGXZK30054) and the High-level Talent Research Start-up Fund of Chizhou University (Grant No. CZ2024YJRC03).

Data Availability Statement: The data and relevant materials presented in this study are openly available at: https://drive.google.com/file/d/172wHj4nZhOh1nFE1J4f4VTj0XEJNtTZL/view?usp=drive_link (accessed on 4 June 2026).

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CI	Continuous Integration
CD	Continuous Delivery
UEM	Unified Event Model
HEG	Heterogeneous execution graph
HGT	Heterogeneous graph Transformer
HB	Happens-before relation
MLP	Multi-layer perceptron
GCN	Graph Convolutional Network
GAT	Graph Attention Network
R-GCN	Relational Graph Convolutional Network
P	Precision
R	Recall
F1	F1-score

References

1. Iannone, E.; Guadagni, R.; Ferrucci, F.; De Lucia, A.; Palomba, F. The secret life of software vulnerabilities: A large-scale empirical study. *IEEE Trans. Softw. Eng.* **2022**, *49*, 44–63. [CrossRef]
2. Krasner, H. The Cost of Poor Software Quality in the US: A 2020 Report. Consortium for Information & Software Quality™. 2021, *2*, 3. Available online: https://www.it-cisq.org/cisq-files/pdf/CPSQ-2020-report.pdf (accessed on 1 May 2026).

3. Ugwueze, V.U.; Chukwunweike, J.N. Continuous integration and deployment strategies for streamlined DevOps in software engineering and application delivery. *Int. J. Comput. Appl. Technol. Res.* **2024**, *14*, 1–24.
4. Lam, W.; Winter, S.; Wei, A.; Xie, T.; Marinov, D.; Bell, J. A large-scale longitudinal study of flaky tests. *Proc. ACM Program. Lang.* **2020**, *4*, 1–29. [CrossRef]
5. Parry, O.; Kapfhammer, G.M.; Hilton, M.; McMinn, P. A survey of flaky tests. *ACM Trans. Softw. Eng. Methodol.* **2021**, *31*, 1–74. [CrossRef]
6. Parry, O.; Kapfhammer, G.M.; Hilton, M.; McMinn, P. Empirically evaluating flaky test detection techniques combining test case rerunning and machine learning models. *Empir. Softw. Eng.* **2023**, *28*, 72. [CrossRef]
7. Lam, W.; Godefroid, P.; Nath, S.; Santhiar, A.; Thummalapenta, S. Root causing flaky tests in a large-scale industrial setting. In Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, Beijing, China, 15–19 July 2019; pp. 101–111.
8. Shabani, T.; Nashid, N.; Alian, P.; Mesbah, A. Dockerfile Flakiness: Characterization and Repair. *arXiv* **2024**, arXiv:2408.05379.
9. Akli, A.; Haben, G.; Habchi, S.; Papadakis, M.; Le Traon, Y. FlakyCat: Predicting flaky tests categories using few-shot learning. In *Proceedings of the 2023 IEEE/ACM International Conference on Automation of Software Test (AST)*; IEEE: Piscataway, NJ, USA, 2023; pp. 140–151.
10. More, R.; Bradbury, J.S. An Analysis of LLM Fine-Tuning and Few-Shot Learning for Flaky Test Detection and Classification. In *Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE: Piscataway, NJ, USA, 2025; pp. 349–359.
11. Barbosa, K.; Ferreira, R.; Pinto, G.; d'Amorim, M.; Miranda, B. Test flakiness across programming languages. *IEEE Trans. Softw. Eng.* **2022**, *49*, 2039–2052. [CrossRef]
12. An, G.; Yoon, J.; Bach, T.; Hong, J.; Yoo, S. Just-in-time flaky test detection via abstracted failure symptom matching. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*; IEEE: Piscataway, NJ, USA, 2024; pp. 741–752.
13. Gruber, M.; Heine, M.; Oster, N.; Philippsen, M.; Fraser, G. Practical flaky test prediction using common code evolution and test history data. In *Proceedings of the 2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE: Piscataway, NJ, USA, 2023; pp. 210–221.
14. Lam, W.; Shi, A.; Oei, R.; Zhang, S.; Ernst, M.D.; Xie, T. Dependent-test-aware regression testing techniques. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2020; pp. 298–311.
15. Ahmad, A.; Held, E.N.; Leifler, O.; Sandahl, K. Identifying randomness related flaky tests through divergence and execution tracing. In *Proceedings of the 2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*; IEEE Computer Society: Washington, DC, USA, 2022; pp. 293–300.
16. Fatima, S.; Hemmati, H.; Briand, L.C. FlakyFix: Using large language models for predicting flaky test fix categories and test code repair. *IEEE Trans. Softw. Eng.* **2024**, *50*, 3146–3171. [CrossRef]
17. Parry, O.; Kapfhammer, G.M.; Hilton, M.; McMinn, P. Surveying the developer experience of flaky tests. In Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, Pittsburgh, PA, USA, 21–29 May 2022; pp. 253–262.
18. Hashemi, N.; Tahir, A.; Rasheed, S. An empirical study of flaky tests in javascript. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*; IEEE: Piscataway, NJ, USA, 2022; pp. 24–34.
19. Parry, O.; Kapfhammer, G.M.; Hilton, M.; McMinn, P. Evaluating features for machine learning detection of order-and non-order-dependent flaky tests. In *Proceedings of the 2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE Computer Society: Washington, DC, USA, 2022; pp. 93–104.
20. Alshammari, A.; Morris, C.; Hilton, M.; Bell, J. Flakeflagger: Predicting flakiness without rerunning tests. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*; IEEE: Piscataway, NJ, USA, 2021; pp. 1572–1584.
21. Fatima, S.; Ghaleb, T.A.; Briand, L. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Trans. Softw. Eng.* **2022**, *49*, 1912–1927. [CrossRef]
22. Qin, Y.; Wang, S.; Liu, K.; Lin, B.; Wu, H.; Li, L.; Mao, X.; Bissyandé, T.F. Peeler: Learning to effectively predict flakiness without running tests. In *Proceedings of the 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*; IEEE: Piscataway, NJ, USA, 2022; pp. 257–268.
23. Pontillo, V.; Palomba, F.; Ferrucci, F. Static test flakiness prediction: How far can we go? *Empir. Softw. Eng.* **2022**, *27*, 187. [CrossRef] [PubMed]
24. Meyer, A.N.; Barr, E.T.; Bird, C.; Zimmermann, T. Today was a good day: The daily life of software developers. *IEEE Trans. Softw. Eng.* **2019**, *47*, 863–880. [CrossRef]

25. Eck, M.; Palomba, F.; Castelluccio, M.; Bacchelli, A. Understanding flaky tests: The developer's perspective. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Tallinn, Estonia, 26–30 August 2019; pp. 830–840.
26. Parry, O.; Gruber, M.; Henderson, T.A.; McMinn, P.; Fraser, G. Summary of the 1st International Flaky Test Workshop (FTW 2024). *ACM SIGSOFT Softw. Eng. Notes* **2024**, *49*, 35–36. [CrossRef]
27. Ziftci, C.; Cavalcanti, D. De-flake your tests: Automatically locating root causes of flaky tests in code at google. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*; IEEE: Piscataway, NJ, USA, 2020; pp. 736–745.
28. Pontillo, V.; Palomba, F.; Ferrucci, F. Test code flakiness in mobile apps: The developer's perspective. *Inf. Softw. Technol.* **2024**, *168*, 107394. [CrossRef]
29. Alshammari, A.; Ammann, P.; Hilton, M.; Bell, J. 230,439 Test Failures Later: An Empirical Evaluation of Flaky Failure Classifiers. In *Proceedings of the 2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE: Piscataway, NJ, USA, 2024; pp. 257–268.
30. Alshammari, A.; Ammann, P.; Hilton, M.; Bell, J. A Study of Flaky Failure De-Duplication to Identify Unreliably Killed Mutants. In *Proceedings of the 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*; IEEE: Piscataway, NJ, USA, 2024; pp. 257–262.
31. Cordy, M.; Rwemalika, R.; Franci, A.; Papadakis, M.; Harman, M. Flakime: Laboratory-controlled test flakiness impact assessment. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 21–29 May 2022; pp. 982–994.
32. Silva, D.; Gruber, M.; Gokhale, S.; Arteca, E.; Turcotte, A.; d'Amorim, M.; Lam, W.; Winter, S.; Bell, J. The effects of computational resources on flaky tests. *IEEE Trans. Softw. Eng.* **2024**, *50*, 3104–3121. [CrossRef]
33. Bell, J.; Legunsen, O.; Hilton, M.; Eloussi, L.; Yung, T.; Marinov, D. DeFlaker: Automatically detecting flaky tests. In Proceedings of the 40th International Conference on Software Engineering, Gethenburg, Sweden, 27 May–3 June 2018; pp. 433–444.
34. Li, C.; Shi, A. Evolution-aware detection of order-dependent flaky tests. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 114–125.
35. Li, C.; Khosravi, M.M.; Lam, W.; Shi, A. Systematically producing test orders to detect order-dependent flaky tests. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2023; pp. 627–638.
36. Wei, A.; Yi, P.; Li, Z.; Xie, T.; Marinov, D.; Lam, W. Preempting flaky tests via non-idempotent-outcome tests. In *Proceedings of the 44th International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 1730–1742.
37. Wang, R.; Chen, Y.; Lam, W. iPFlakies: A framework for detecting and fixing python order-dependent flaky tests. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 120–124.
38. Li, C.; Zhu, C.; Wang, W.; Shi, A. Repairing order-dependent flaky tests via test generation. In *Proceedings of the 44th International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 1881–1892.
39. Rahman, S.; Shi, A. FlakeSync: Automatically repairing async flaky tests. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 1–12.
40. Chen, Y. Flakiness repair in the era of large language models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 441–443.
41. Chen, Y.; Jabbarvand, R. Neurosymbolic repair of test flakiness. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 1402–1414.
42. Leesatapornwongsa, T.; Ren, X.; Nath, S. FlakeRepro: Automated and efficient reproduction of concurrency-related flaky tests. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 1509–1520.
43. Zhang, L.; Miranskyy, A. Automated flakiness detection in quantum software bug reports. In *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*; IEEE: Piscataway, NJ, USA, 2024; Volume 2, pp. 179–181.
44. Amini, M.H.; Naseri, S.; Nejati, S. Evaluating the impact of flaky simulators on testing autonomous driving systems. *Empir. Softw. Eng.* **2024**, *29*, 47. [CrossRef]
45. Gruber, M.; Roslan, M.F.; Parry, O.; Scharnböck, F.; McMinn, P.; Fraser, G. Do automatic test generation tools generate flaky tests? In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 1–12.
46. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. Codebert: A pre-trained model for programming and natural languages. In *Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 1536–1547.

47. Lam, W.; Oei, R.; Shi, A.; Marinov, D.; Xie, T. iDFlakies: A framework for detecting and partially classifying flaky tests. In *Proceedings of the 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*; IEEE: Piscataway, NJ, USA, 2019; pp. 312–322.
48. Wei, A.; Yi, P.; Xie, T.; Marinov, D.; Lam, W. Probabilistic and systematic coverage of consecutive test-method pairs for detecting order-dependent flaky tests. In *Tools and Algorithms for the Construction and Analysis of Systems: 27th International Conference, TACAS 2021*; Springer: Cham, Switzerland, 2021; pp. 270–287.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.